

An Agentic Task Offloading Workflow Method for Intellicise Wireless Networks in the Industrial Internet of Things

1st Guowei Ni

Jiangsu University of Technology
Changzhou, China
ZGC Institute of Ubiquitous-X Innovation and
Applications
Beijing, China
niguowei44@163.com

2nd Weichen Ni

China Mobile Research Institute
China Mobile Communications Group Co., Ltd
Beijing, China
niweichen@chinamobile.com

3rd Boxiao Han

ZGC Institute of Ubiquitous-X Innovation and
Applications
Beijing, China
hanboxiao@zgc-xnet.com

4th Zuojun Dai

ZGC Institute of Ubiquitous-X Innovation and
Applications
Beijing, China
george.dai@bupt.edu.cn

5th Ziyang Jia*

Jiangsu University of Technology
Changzhou, China
jjiaziyan@jsut.edu.cn

Abstract—To address the challenge of inaccurate intent recognition in Intellicise (intelligent and concise) Wireless Networks-enabled edge-cloud-terminal computing scenarios, which utilize large language models (LLMs) for task offloading and resource allocation, we propose a two-tier multi-agent workflow method. This method employs slot-filling technology in the Intention Layer to parse user requests and subsequently coordinates multiple agents in the Efficiency Layer to collaboratively optimize scheduling policies by converting subjective intentions into measurable utility metrics. LLMs perceive key factors—such as device location, task type, and context—and adjust task allocation parameters dynamically. They then guide agents to use appropriate tools and perform optimized scheduling based on the task's nature. Experimental results demonstrate that our method accurately recognizes user intents by considering multi-dimensional factors such as edge device deployment-related regional economic factors, network conditions, and IoT device metrics, thereby meeting diverse user intent.

Keywords—Industrial Internet of Thing; Agentic Workflow; Multi-Agent System; LLMs

I. INTRODUCTION

The revolutionary advancements of 5G have transformed industrial production. Yet their inability to meet ultra-low latency and high-reliability demands in dynamic Industrial Internet of Things (IIoT) environments has emerged as a critical bottleneck for intelligent applications [1]. In this context, the Intellicise Wireless Networks — characterized by inherent intelligence and simplified architecture—provide opportunities and challenges for optimizing edge computing in IIoT through AI-empowered future 6G networks [2]. However, accurately interpreting user intent via AI to ensure alignment among user requirements, dynamically evolving resource allocation demands, and real-time quality of

service (QoS) specifications remain a complex challenge. Traditional task offloading methods, constrained by static policies and limited adaptability to the stochastic nature of task data variability and latency requirements, often lead to suboptimal utilization of edge-cloud server resources and excessive energy consumption [3]. Recently, advancements in multi-agent technologies, such as Manus, have introduced novel solutions to address these challenges. These agents integrate autonomous learning, decision-making, and execution capabilities, enabling dynamic adaptation to environmental fluctuations and task-specific constraints. Deploying collaborative multi-agent workflows enables systems to monitor real-time network states and dynamically optimize resource allocation based on user intent. This approach not only enhances the efficiency of edge computing resource utilization but also significantly reduces the overall energy consumption of IIoT systems [4].

Effective task offloading fundamentally relies on the precise interpretation of user intentions. However, even with extensive training, LLMs frequently exhibit limitations in processing nuanced and domain-specific content, such as IIoT scenarios, potentially leading to user intent misinterpretations. To address this, Zhang et al. [5] proposed a few-shot intent detection framework integrating contrastive learning with correlation matrix regularization, demonstrating improved accuracy in intent recognition for heterogeneous task requirements. In parallel, Ren et al. [6] developed the Enhanced Inten Recognition Accuracy based on Domain Knowledge (EIRDK) method, which refines LLM performance in specialized question-answering applications through prompt normalization, intent scoring hierarchies, and domain-specific word vector alignment. While EIRDK achieves notable precision, its efficacy critically depends on the availability of high-quality domain knowledge bases, introducing vulnerabilities due to potential inaccuracies or incompleteness in such resources.

Further advancing this field, Zhang et al. [7] designed a graph attention network-based joint model (WISM) for multi-intent recognition and semantic slot filling, enabling fine-grained mappings between lexical units, intents, and semantic roles through bidirectional word-level modeling. Despite its innovations, WISM overlooks deeper semantic analyses—such as entity-relationship extraction—that could further enhance intent understanding in complex IIoT workflows.

The emergence of agentic workflow paradigms has spurred innovative solutions for intent-aware systems. In this context, Li et al. [8] proposed a novel termed Large-Language-Models Enhanced Intent-Aware Mobility Prediction (LLM-IAMP), which employs an Analyze-Abstract-Infer (A2I) agentic workflow to mitigate inaccurate intent recognition in LLMs through intent feature analysis, insight summarization, and user intent inference. However, each step of the A2I workflow relies on the reasoning capabilities of LLMs, and suboptimal LLM performance may compromise the overall predictive accuracy of the workflow.

LLMs can enhance IIoT task offloading but often struggle with complex inputs, leading to challenges such as hallucination phenomena and misinterpreted user intent, which degrade offloading outcomes [9]. To address these challenges, we propose a two-tier architecture integrating an Intention Layer and an Efficiency Layer, supported by a hybrid agent-based workflow that synergizes human-agent collaboration and multi-agent coordination. This architecture dynamically adapts to input complexity and system constraints, enabling accurate user intent recognition in tasks such as task offloading and resource optimization.

The agent in the Intention Layer acquires societal, regional, organizational, and business context knowledge through human-agent interactions and its autonomous capabilities, enabling precise interpretation of user intent. This process involves decomposing complex tasks and generating logically structured plans [10]. In parallel, the Efficiency Layer facilitates collaborative agent operations—including task specification, scheduling strategies formulation, and execution management,—guided by performance metrics, feedback, and contextual data analysis. The contributions of this paper are as follows:

1. To address the limitations of LLMs in intent recognition for AI-powered IIoT task offloading scenarios, we propose a two-tier architecture agentic workflow method, which is structured around Intention and Efficiency layers. The proposed method synergizes human-agent interaction and multi-agent collaboration, dynamically integrating network conditions monitoring and resource utilization metrics to achieve precise intent recognition.

2. Within the agentic workflow, an evaluation agent is utilized to assess scheduling outcomes based on the network conditions and resource efficiency of edge devices. When required, a structured sequence of agents guides re-execution processes to refine task allocation and scheduling optimization through iterative feedback loops.

3. By using slot-filling technology to improve the agent's accuracy in user intent recognition, thereby achieving efficient task scheduling.

The remainder of this paper is organized as follows. Section II presents a detailed explanation of the proposed multi-agent workflow. Section III details the experimental validation of the proposed method. Section IV concludes the study and proposes future research directions to further enhance the framework.

II. SYSTEM AND WORKFLOW

A. System Description

In current research, agents are typically provided with complete and detailed task descriptions as initial input at the outset. However, this non-interactive assumption is uncommon in real-world task planning [11]. Inspired by papers [12] and [13], we combine human-agent interaction with multi-agent collaboration to design a multi-agent workflow method, which is applied in the two-tier system architecture shown in Fig. 1. This system comprises two distinct layers: the Intention Layer and the Efficiency Layer. The Intention Layer interprets user requirements to generate task plans by analyzing contextual inputs, while the Efficiency Layer optimizes resource allocation through multi-dimensional considerations, including regional economic conditions, cultural contexts, political environments, and enterprise operational metrics.

B. Slot Mecahnism

Current slot-filling methods primarily rely on rule-based, statistics-based, and deep learning-based approaches [14]. In the proposed two-tier architecture, LLMs are leveraged to execute slot-filling tasks within the Intention Layer. We consider three regions at different development stages: developed, middle-developed, and developing. Region-specific slot templates are predefined for each category, specifying parameters such as latency and energy consumption preferences. The dialogue is divided into two modes: closed-domain and open-domain multi-turn dialogue [15]. Closed-domain dialogues operate within predefined scenarios, while open-domain dialogues allow broader interactions. For open-domain conversations, the LLM identifies the user's scenario and transitions to closed-domain dialogue if a match is found. The LLM retrieves the corresponding template and maps it to the input, completing slot filling when all parameters are met or prompting the user for missing information. To handle intent shifts, the system reassesses the scenario at each turn, ensuring alignment with user needs.

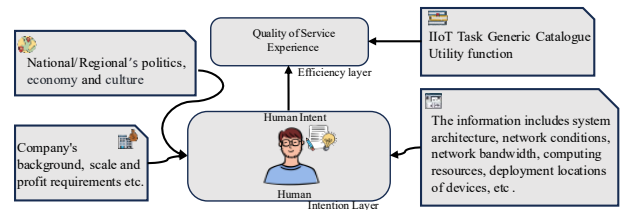


Figure 1. The two-tier system architecture.

C. Agentic Workflow

Cemri et al. [16] identified several reasons for agent failures, including the inability to request clarification from users, loss of conversation history, and lack of awareness of termination conditions. Based on Cemri's theoretical

analysis, we implemented targeted improvements. From a theoretical perspective, addressing these issues enables the multi-agent workflow to process tasks correctly. TABLE I outlines the four phases of the workflow—requirement analysis, planning and execution, scheduling, and evaluation—as well as the agents involved in each phase.

All agents illustrated in Fig. 2 are driven by LLMs. These agents utilize the text analysis capabilities of LLMs to parse user intent and analyze key factors such as the geographic distribution of terminal devices, task characteristics, and contextual information, dynamically generating parameters for the utility function. In task offloading experiments, since different tasks have varying requirements in terms of latency and energy consumption, the LLM adjusts system parameters. It drives the agents to calculate the minimum latency and energy consumption required to complete the current task. Based on this result, the agent decides whether the task should be processed locally or offloaded to edge devices. In resource allocation experiments, the LLM dynamically adjusts system parameters based on scheduling outcomes and the deviation between user requirements and the system's lower bounds. The agent then invokes tools to change the number of subcarriers allocated to users at different distances within the current time frame, satisfying user demands and the system's minimum requirements.

The functions of different agents are as follows:

- The Account Manager Agent interacts with users, identifies dialogue contexts, ensures slot

completeness, and requests missing parameters. Once all required slots are filled, it processes user information to formulate and decompose complex tasks into sub-tasks.

- The System Architect Agent handles task planning and leverages Retrieval-Augmented Generation technology to gather expert knowledge. It reviews previous scheduling decisions and outcomes from other agents, considering network status and resource usage. Based on this analysis, it advises the Scheduler Agent on user scheduling adjustments.
- The Scheduler Agent integrates outputs from the OAM Agent, the System Architect Agent's analysis, user input, and past scheduling data to finalize scheduling parameters and initiate simulation tools.
- The OAM Agent gathers system network status, including overall utilization, and analyzes wireless bandwidth, computing resources, and terminal resource consumption post-scheduling to assess the Scheduler Agent's outcomes, provide suggestions, and determine the next execution steps.

TABLE I. AGENTS INVOLVED IN DIFFERENT STAGES

Stage	Participating Agents
Requirement analysis	Human, Account Manager Agent
Planning and Execution	Account Manager Agent, System Architect Agent, OAM Agent
Scheduling	System Architect Agent, Scheduler Agent
Evaluation	Scheduler Agent, OAM Agent

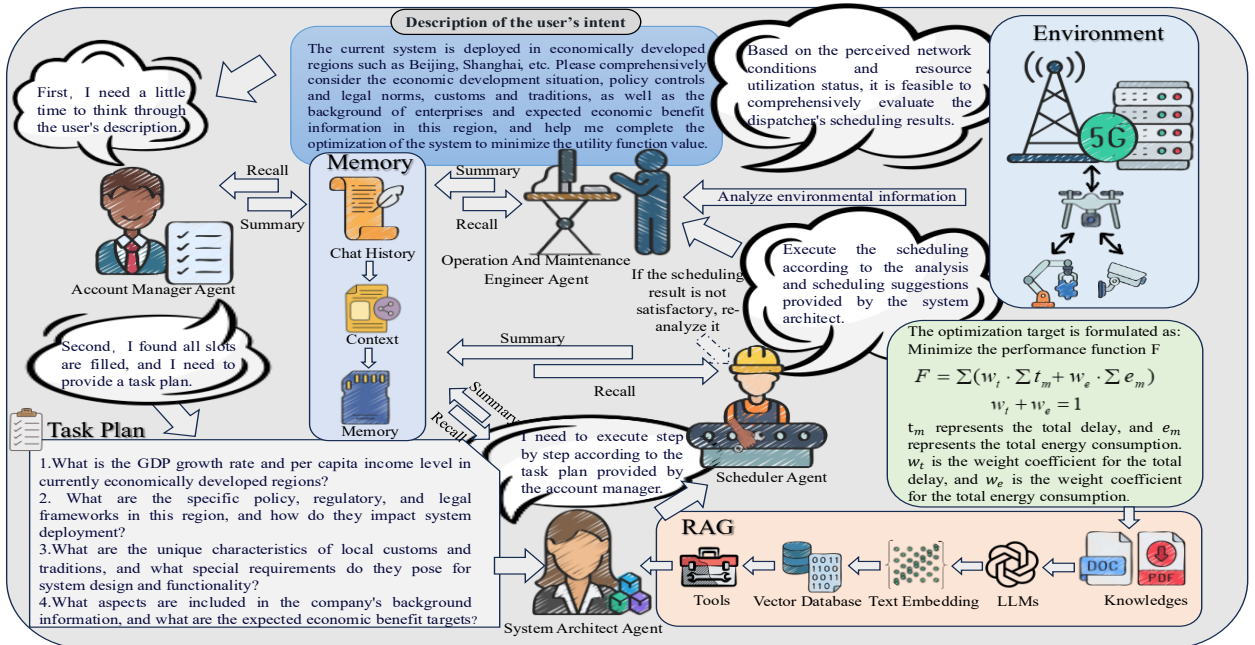


Figure 2. Multi-Agent First Interaction Diagram.

III. EXPERIMENTAL PROCEDURE AND ANALYSIS

A. UAV Task Offloading Experiment

In the unmanned aerial vehicle (UAV) task offloading experiment, the proposed multi-agent workflow is validated through the scenario illustrated in Fig. 3, where computational tasks are offloaded from terminal devices to

UAVs. In this setup, all terminal devices and UAVs are equipped exclusively with CPU computational units, and the communication model follows a terminal-to-UAV protocol. The objective of the agents is to dynamically generate optimal task strategies based on specific task requirements, device states, and environmental conditions. This process aims to optimize the utility function F defined in (1), minimizing the system's total latency and energy consumption while satisfying the constraints outlined in (2).

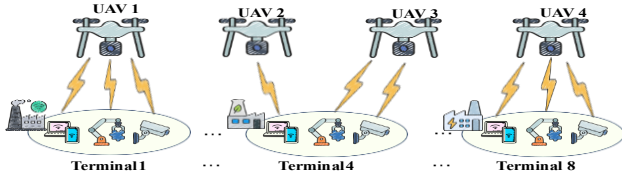


Figure 3. Energy Enterprise Terminals - UAVs Task Offloading.

$$F = \sum(w_t \cdot \sum t_m + w_e \cdot \sum e_m) \quad (1)$$

$$w_t + w_e = 1 \quad (2)$$

Where t_m is the total delay required for task offloading, transmission, and computation, which must be minimized during utility optimization. e_m denotes the total energy consumption during task execution, requiring the agent to balance energy usage between terminal devices and battery-constrained drones to prolong device operation. w_t and w_e are the weight coefficient for delay and energy consumption, respectively. The agent must also balance resource utilization to avoid over-reliance on a single layer's computing resources, preventing bottlenecks or failures. To simplify communication distance calculations, UAVs and terminal positions were fixed. Other specific parameters are listed in TABLE II.

The experiment was conducted across three regions with distinct levels of economic development to evaluate the multi-agent workflow. Fig. 4a and Fig. 4b illustrate the test results for latency and energy consumption achieved by the multi-agent workflow in response to user intent for different regions. The results demonstrate that the multi-agent system dynamically assigns personalized weight values for w_t and w_e based on regional characteristics, local network conditions, resource utilization, and corporate backgrounds, thereby addressing the differentiated needs of businesses in various regions. In developed regions, characterized by well-developed infrastructure, superior communication quality, and strict regulatory oversight on energy consumption, the agents allocate higher w_e weights to prioritize energy efficiency. In middle-developed regions, where infrastructure is moderately developed and regulatory oversight on energy consumption is less stringent, the agents balance priorities by assigning intermediate w_t and w_e weights. In developing regions, where infrastructure is inadequate but governments encourage economic growth through investment and relaxed energy consumption regulations, the workflow increases the w_e weight to

prioritize latency reduction, even at the cost of higher energy expenditure.

The Scheduler Agent, using tools and based on weight results, obtains the weighted sum of latency and energy in Fig. 5a. In developed regions, characterized by robust infrastructure and low network latency, the agent adjusts the weight of w_e , leading to a decrease in the total energy consumption for task offloading. Conversely, in middle-developed regions, agents consider the more comprehensive infrastructure and the less stringent control over energy consumption, aiming to foster investment. Consequently, it achieves a balance between energy consumption and latency for task offloading, seeking to optimize the trade-off between overall latency and energy consumption. In developing regions, where local infrastructure is inadequate and communication latency is pronounced, the model increases the weight of latency. This adjustment results in an elevation of the weighted sum of total latency and energy consumption, thereby increasing energy expenditure to mitigate communication latency. This adaptive mechanism ensures optimal trade-offs between latency and energy consumption tailored to regional requirements.

We also tested the convergence speed of our multi-agent workflow method across various base models in developed regions. Fig. 5b demonstrates DeepSeek-R1-7B converged fastest with the strongest adaptability after 8 rounds, followed by Qwen2.5-7B in 13 rounds. Qwen2-7B and GLM3-6B both required 16 rounds, but Qwen2-7B continuously adjusted parameters using new training techniques before convergence while GLM3-6B stalled between rounds 6-12. Gemma:7B needed 17 rounds. These results confirm that our multi-agent workflow method is effective across different models.

TABLE II. SIMULATION PARAMETER SETTINGS

Parameter	Value
Number of UAVs	4
UAV Working Distance	100m
Areas	400m*400m
Terminal Task Data Volume	5-10 MB
Number of Terminal Devices	8
Number of CPU Cores on Terminals	4
Number of CPU Cores on UAVs	16
Terminal CPU Frequency	0.4 GHz
UAV CPU Frequency	4 GHz
Bandwidth from UE to UAV	1 MHZ
Carrier Frequency	2.6 GHZ
White Gaussian noise	-127 dB

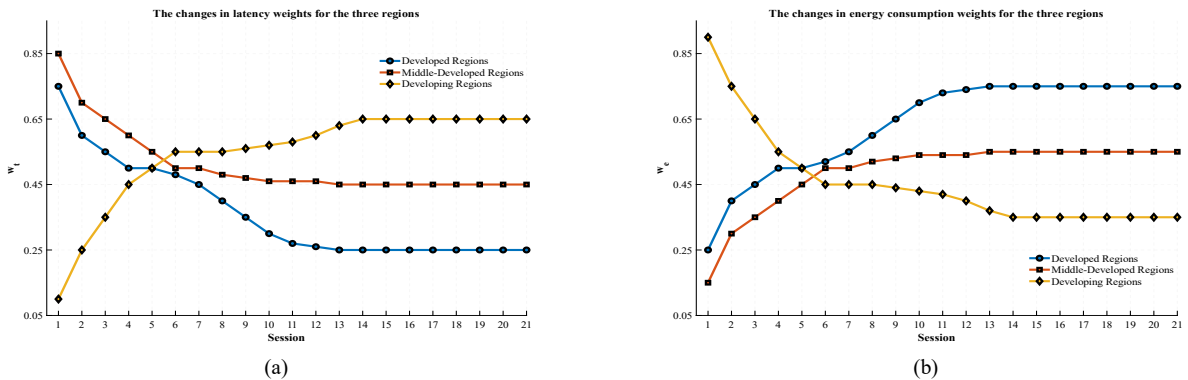


Figure 4. (a) and (b) illustrate the evolution of w_t and w_e across interaction rounds for different regions.

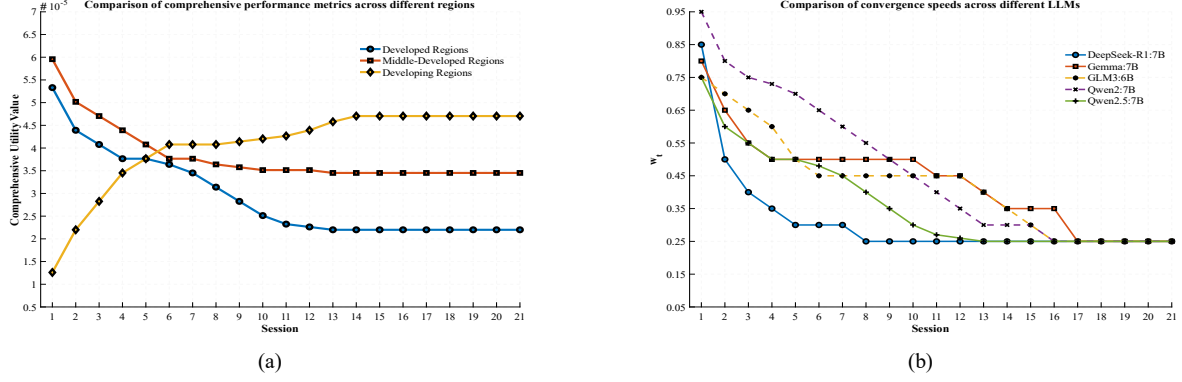


Figure 5. (a) The system's comprehensive performance under different weight parameters.(b) The convergence speed of w_i for different models.

B. Network Resource Optimization Experiment

As illustrated in Fig. 6, the experimental scenario considers a 20×20 km² square region. Within this area, 50 IoT devices are spatially distributed following a homogeneous Poisson Point Process (PPP) with random initialization. Each device generates data packets governed by a Poisson process parameterized with λ , yielding an aggregate average data rate of 100 kbit/s. The base station (BS) is positioned at the upper-right corner of the region. Based on the Euclidean distance between devices and the BS, the devices are categorized into four QoS classes: High-risk group: Top 10% farthest (cumulative 10%), Sub-high-risk group: Next 15% farthest (cumulative 25%), Medium-risk group: Subsequent 25% medium-distance (cumulative 50%), and Low-risk group: Bottom 50% closest.

In wireless propagation models, the increase in Euclidean distance between edge devices and the BS induces exponential growth in path loss, resulting in a lower maximum allowable scheduling time window. Given that the number of subcarriers N in the Orthogonal Frequency Division Multiple Access (OFDMA) system is smaller than the number of devices M , resource contention can lead to scheduling timeouts for some devices, consequently increasing overall system latency. For instance, decreasing the weight value assigned to a specific user category can reduce latency and enhance revenue for that group. Conversely, this adjustment may increase latency and diminish revenue for other user categories. TABLE III lists the parameter settings for the network resource optimization experiment.

The proposed multi-agent workflow method dynamically allocates subcarrier resources to minimize overall system latency while avoiding excessive resource allocation to low-risk group devices. This approach enhances resource utilization efficiency by reducing resource wastage and balancing priority across QoS classes.

Fig. 7 shows simulation results when user-expected reliability is prioritized. Under resource constraints, the multi-agent workflow reduces average latency to 17.9 ms by increasing the risk weights of IoT devices farthest from the BS and reducing weights for those closest to the BS. In contrast, Fig. 8 illustrates the scenario where profitability is prioritized. Here, the multi-agent workflow allocates more resources to devices with poorer communication quality to keep latency between 22.9 ms and 23.7 ms while increasing

weights for devices with better quality to maximize revenue by transmitting larger data volumes.

TABLE III. SIMULATION PARAMETER SETTINGS

Parameter	Value
N	32
M	50
Carrier frequency	2 GHz
Bandwidth	32 kHz
Device transmit power	23 dBm
Noise power	-95 dBm
Minimum receiver SINR	-8 dB

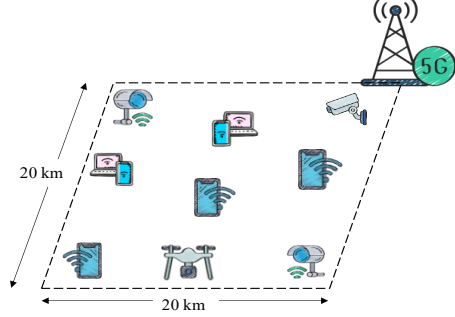


Figure 6. Base Station-IoT Device Communication Scenario.

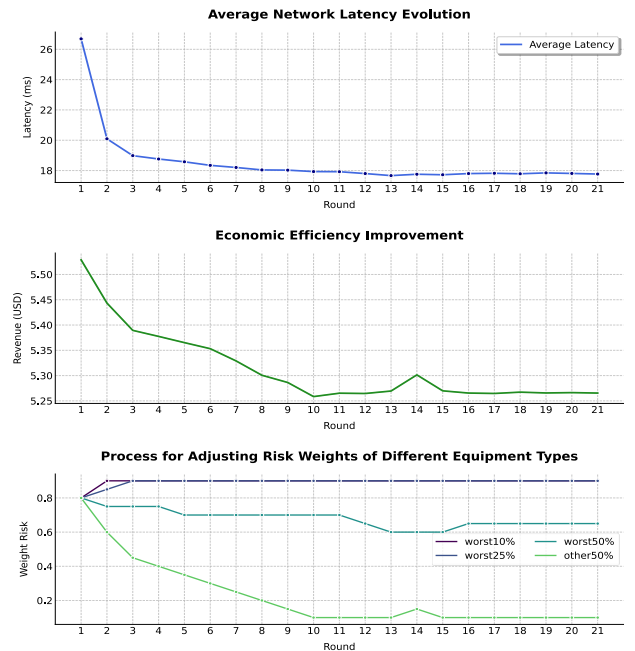


Figure 7. Reliability is prioritized as the primary objective.

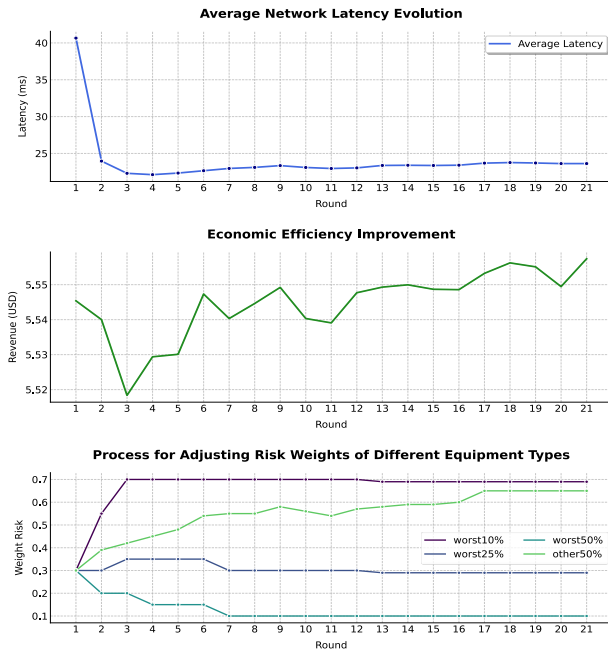


Figure 8. Profitability is prioritized as the primary objective.

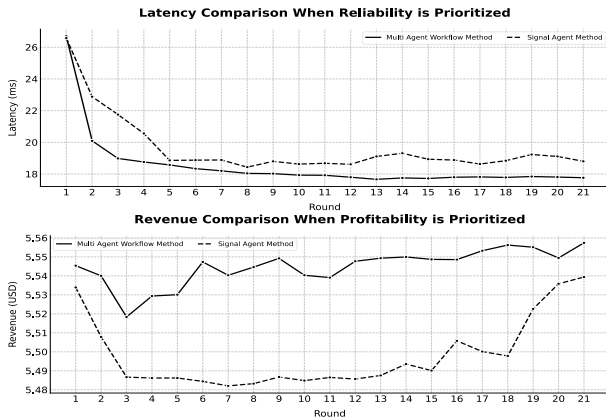


Figure 9. Comparison of latency and revenue with different methods.

To validate the proposed method's advantages, we compared the multi-agent workflow with single-agent approaches. As Fig. 9 shows, the multi-agent method achieves higher system revenue when maximizing profit and lower average delay when prioritizing reliability.

IV. CONCLUSION

This paper proposes a multi-agent workflow method aimed at jointly optimizing task offloading and resource allocation in IIoT scenarios. The architecture integrates human-machine interaction and inter-agent collaboration, combining intent recognition with efficiency optimization. It leverages the retrieval and generation capabilities of LLMs to assist agents in understanding and responding to natural language instructions. Through a two-tier architecture, the system maps user intents to performance metrics. Experimental results demonstrate that this approach effectively enhances the quality of scheduling decisions in task offloading scenarios by comprehensively considering regional economic factors of edge-cloud device deployment, system network conditions, and resource utilization of edge devices. Additionally, even under

constrained communication environments, the method holistically addresses IoT device resource requirements and operational states, enabling tailored resource allocation aligned with user-specific intents. However, due to constraints imposed by the initial dual-layer network structure, the full potential of the multi-agent system remains underutilized. Future work could expand the system to a four-tier architecture, incorporating business regulations and technical specifications of edge-cloud devices to achieve more granular and optimized scheduling.

ACKNOWLEDGMENT

This work is funded by China Mobile Communications Group Co., Ltd.

REFERENCES

- [1] P. Zhang, X.D. Xu, et al Lan, Entropy Reduced Mobile Networks Empowering Industrial Applications, *Journal of Beijing University of Posts and Telecommunications*, vol. 43, Dec. 2020, pp. 1-9.
- [2] Y.H. Huang, Q.X. Wang, N. Li, Intelligent and Lean 6G Network, *ZTE technology journal*, vol. 30, Apr. 2024, pp. 3-9.
- [3] Y.J. Xu, B.W. Gu, Q.B. Chen, Energy Consumption Optimization Algorithm for Full-Duplex Relay-Assisted Mobile Edge Computing Systems, *Journal of Electronics & Information Technology*, vol. 43, Dec. 2021, pp. 3621-3628.
- [4] F. Zhu, F. Huang, Y. Yu, et al. Task Offloading with LLM-Enhanced Multi-Agent Reinforcement Learning in UAV-Assisted Edge Computing, *Sensors*, vol. 25, Nov. 2024, pp. 1-19.
- [5] H.D. Zhang, H.W. Liang, Y.W. Zhang, L.M. Zhan, X.L. Lu, et al., Fine-tuning Pre-trained Language Models for Few-shot Intent Detection: Supervised Pre-training and Isotropization, In: *NAACL 2022*. Seattle, United States. Jul. 2022, pp. 532-542.
- [6] Y.K. Ren and Z.P. Xie, Intention recognition accuracy enhancing method for large language model, *Application Research Of Computers*, vol. 42, Jan. 2025, pp. 2893-2899.
- [7] Y.H. Zhang, L. Chen, S.G. Ju, M.W. Li, Joint Model for Multi-intent Detection and Slot Filling Based on Graph Attention Network. *Journal of Software*, vol. 35, Dec. 2024, pp. 5509-5525.
- [8] S.W. Li, F. Jie, J.W. Chi, X.Y. Hu, X.M. Zhao, F.L. Xu, LIMP: Large Language Model Enhanced Intent-aware Mobility Prediction, *arXiv preprint*, Aug. 2024.
- [9] M. Sudarshan, S. Shih, E. Yee, A. Yang, et al, Agentic LLM Workflows for Generating Patient-Friendly Medical Reports, *arXiv preprint*, Aug. 2024.
- [10] A. Saparov, R.Y. Pang, V. Padmakumar, N. Joshi, S.M. Kazemi, N. Kim, H. He, Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples, In: *Advances in Neural Information Processing Systems*. New Orleans, United States. Dec. 2023.
- [11] R.X. Xiao, W.T. Ma, K. Wang, Y.C. Wu, J.B. Zhao, H.B. Wang, F. Huang, Y.B. Li, FlowBench: Revisiting and Benchmarking Workflow-Guided Planning for LLM-based Agents, In: *Findings of the ACL: EMNLP 2024*, United States. Jul. 2022, pp. 10883-10900.
- [12] Y.Y. Wang, X.J. Li, B.Z. Wang, Y.Y. Zhou, et al. PEER: Expertizing Domain-Specific Tasks with a Multi-Agent Framework and Tuning Methods, *arXiv preprint*, Jul. 2024.
- [13] C. Qian, W. Liu, H. Liu, et al. ChatDev: Communicative Agents for Software Development, In: *Proceedings of the 62nd Annual Meeting of the ACL*, Bangkok, Thailand. Aug. 2024, pp. 15174-15186.
- [14] H. Weld, X.Q. Huang, S.Q. Long, J. Poon, S.C. Han, A Survey of Joint Intent Detection and Slot Filling Models in Natural Language Understanding, *ACM Computing Surveys*, vol. 55, Dec. 2022, pp. 1-38.
- [15] A. Sulayman, M.S.H. Salam, F. Mohamed, A Review on Approaches in Arabic Chatbot for Open and Closed Domain Dialog, *International Journal of Advanced Computer Science and Applications*, vol. 13, Nov. 2022, pp. 158-167.
- [16] M. Cemri, M.Z. Pan, S. Yang, et al. Why Do Multi-Agent LLM Systems Fail?, *arXiv preprint*, Apr. 2025.